

PARTNER PORTAL

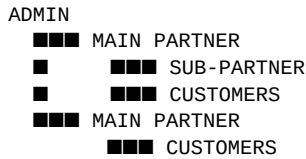
Master Project Documentation

Version 1.0 • Ready for Implementation

Backend	Laravel 13
PHP	8.3+
Database	MySQL 8.x
Frontend	Inertia.js + Vue 3 + Vite
Redis	Not used initially
MVP	Partner Portal core functionality
Phase 2	Customer Portal + Commission Management

1. Product Overview

Partner Portal is a web-based application for a company that sells products/services through Main Partners and Sub-Partners. The primary application users are Admin, Main Partner and Sub-Partner. A dedicated Customer Portal is Phase 2.



2. Project Goals

Admin

- Manage Partners and Sub-Partners
- Manage customers, leads, products/plans, subscriptions and payments
- Manage referral codes and Partner mapping
- Monitor renewals
- View reports, notifications and audit history

Main Partner

- Manage own leads/customers/sales
- View subscriptions and renewals
- Create and manage Sub-Partners
- View Sub-Partner performance
- View reports and notifications

Sub-Partner

- Manage own leads/customers/sales
- View own subscriptions and renewals

- View reports and notifications
- Cannot access another Sub-Partner's customers or sales

3. MVP and Phase 2

MVP

- Authentication and security
- Partner/Sub-Partner management
- Referral system
- Lead management
- Customer management
- Products and plans
- Orders/sales
- Payment records
- Subscriptions
- Auto-debit
- Renewals
- Notifications
- Dashboards
- Reports
- Audit logs

Phase 2

- Customer Portal
- Commission Management
- Commission calculation/approval/payment workflow

4. Confirmed Business Decisions

Area	Decision
Customer Portal	Phase 2
Commission	Phase 2
Auto-debit stop	Admin only
Existing customer clicks another referral	Keep existing Partner
Registration without referral	Unassigned
Attribution persistence	Until Admin changes it
Duplicate customer	Email + Mobile
Sub-Partner creates Sub-Partner	No
Sub-Partners	Unlimited
Historical sales after mapping change	Original Partner
Future sales after mapping change	New Partner

Area	Decision
Main/Sub Partner subscription changes	No
Partner cancellation	No
Referral URL	/register?partner=CODE
Products/Plans	Admin manages
Payment gateway	TBD
Notifications	Email + In-App
Reports	Excel + CSV + PDF
Responsive	Desktop + Tablet + Mobile
Redis	No initially

5. Core Business Rules

- Referral attribution happens automatically.
- Once attributed, a customer remains with the Partner unless Admin changes the mapping.
- Existing customers do not move automatically when they use another Partner's referral link.
- Registration without a referral creates an unassigned customer/lead.
- Customer identity is normalized Email + Mobile.
- Only Main Partners can create Sub-Partners; Sub-Partners cannot create Sub-Partners.
- Historical sales remain with the original Partner after a mapping change.
- Future sales use the new Partner mapping.
- Deactivated Partners do not receive new leads.
- Historical ownership and payment records must remain available.
- Every important business action must have an audit trail.

6. Technology Architecture

```

Browser
  ↓
Inertia.js + Vue 3 + Vite
  ↓
Laravel 13
  ■■■ Controllers
  ■■■ Policies
  ■■■ Actions / Domain Services
  ■■■ Jobs / Events / Listeners
  ■■■ Notifications
  ■■■ Scheduler
  ↓
MySQL 8.x

```

Redis is intentionally excluded from the initial architecture. MySQL remains the source of truth.

7. Laravel Architecture

```

app/
■■■ Actions/
■■■ Domain/
■   ■■■ Authentication/
■   ■■■ Partners/
■   ■■■ Referrals/
■   ■■■ Leads/

```

- ■■■ Customers/
- ■■■ Products/
- ■■■ Subscriptions/
- ■■■ Payments/
- ■■■ Renewals/
- ■■■ Notifications/
- ■■■ Reports/
- ■■■ Audit/
- Http/
- ■■■ Controllers/
- ■■■ Requests/
- ■■■ Resources/
- ■■■ Middleware/
- Models/
- Policies/
- Jobs/
- Events/
- Listeners/
- Notifications/
- Services/
- Support/

Controllers should validate, authorize and delegate. Complex business logic belongs in Actions/Domain services.

8. Database — Core Tables

- users, partner_users
- partners, referral_codes, customer_partner_attributions
- customers, leads, lead_status_histories, lead_follow_ups
- products, product_plans
- orders, order_items
- payments, payment_transactions
- subscriptions, subscription_status_histories
- auto_debit_mandates, auto_debit_events
- renewals
- notifications, audit_logs
- Phase 2: commissions, commission_status_histories, commission_payments

9. Database ERD

```

users
  ■
  ■■■ partner_users ■■ partners
    ■
    ■■■ referral_codes
    ■■■ leads
    ■■■ customers
      ■■■ customer_partner_attributions
      ■■■ orders ■■ order_items
      ■■■ subscriptions
        ■■■ status_history
        ■■■ auto_debit_mandates ■■ events
        ■■■ renewals

orders ■■ payments ■■ payment_transactions

Important entities ■■ audit_logs
  
```

10. Ownership and Historical Data

Current customer mapping is stored on customers.current_partner_id/current_sub_partner_id. Historical mapping is stored in customer_partner_attributions. Orders and subscriptions retain Partner/Sub-Partner ownership snapshots so historical reporting is not changed when Admin changes the customer's current mapping.

Customer

- Current mapping → Partner B
- Attribution history
 - Partner A (old)
 - Partner B (current)

Order before mapping change → Partner A

Order after mapping change → Partner B

11. Critical Database Constraints

- partner_code UNIQUE
- referral code UNIQUE
- customer_code UNIQUE
- order_number UNIQUE
- subscription_number UNIQUE
- product.code UNIQUE
- (product_id, code) UNIQUE for product plans
- (email_normalized, mobile_normalized) UNIQUE for customers
- Monetary fields use DECIMAL(15,2) and must not be negative.

12. Delete Strategy

Do not cascade-delete core business history. Prefer ON DELETE RESTRICT for historical/business relationships and ON DELETE SET NULL where a relationship is genuinely optional. Partner deactivation uses status rather than deleting the Partner and its history.

13. Authorization and Data Isolation

Authentication

↓

Role

↓

Policy

↓

Data Scope

↓

Action

Authorization is server-side. Frontend restrictions are not security boundaries. Sub-Partner access must be scoped to the authenticated Sub-Partner, preventing IDOR/manual-ID access to another Sub-Partner's records.

14. Transaction Boundaries

Registration

```
BEGIN
Resolve referral
Find/Create Customer
Create Attribution
Create Lead
COMMIT
```

Lead Conversion

```
BEGIN
Order
Order Items
```

Payment
Customer
Subscription
Attribution
COMMIT

Partner Mapping Change

BEGIN
Lock Customer
Close old attribution
Create new attribution
Update current mapping
Create Audit
COMMIT

15. Referral Flow

Referral link
↓
Registration
↓
Resolve referral code
↓
Find/Create customer
↓
Create attribution
↓
Create lead
↓
Partner sees lead

Referral URL: /register?partner=CODE

16. Lead Lifecycle

New → Contacted → Demo Scheduled → Demo Completed
→ Proposal Sent → Negotiation → Payment Pending → Converted

Alternative: Lost / Not Interested

17. Sales and Payment

Lead
↓
Purchase
↓
Payment
↓
Order Created
↓
Customer
↓
Subscription Activated

Payment gateway is intentionally TBD. The application should use a PaymentGateway abstraction so a provider can be introduced later without redesigning the core payment model.

18. Subscription and Auto-Debit

Order
↓
Payment confirmed
↓
Subscription activated
↓
Active
↓
Upcoming Renewal
↓

Payment Pending



Renewed

- Admin can change/cancel subscriptions.
- Main Partner and Sub-Partner can view but cannot change/cancel subscriptions.
- Only Admin can stop auto-debit.
- Every auto-debit change is recorded in auto_debit_events and audit_logs.

19. Renewals

30 days → Optional reminder

15 days → Partner + Customer

7 days → Partner + Customer

1 day → Final reminder

Renewal Due



Partner contacts customer



Customer confirms



Payment



Payment confirmed



Subscription renewed



New expiry date

20. Notifications

- Email + In-App
- New Lead
- New Customer
- Sale Completed
- Payment Received
- Renewal Due
- Payment Failed
- Auto-Debit Stopped
- Subscription Cancelled
- Sub-Partner Created

21. Reports

- Sales Report
- Customer Report
- Renewal Report
- Excel export
- CSV export
- PDF export

22. Security

- RBAC
- Server-side authorization
- Password hashing
- OTP expiry
- Session/token expiry
- Login rate limiting
- Audit logging
- Sensitive-data protection
- IDOR protection
- Partner/Sub-Partner data isolation

23. Migration Order

```
01 users
02 partners
03 partner_users
04 products
05 product_plans
06 referral_codes
07 customers
08 customer_partner_attributions
09 leads
10 lead_status_histories
11 lead_follow_ups
12 orders
13 order_items
14 payments
15 payment_transactions
16 subscriptions
17 subscription_status_histories
18 auto_debit_mandates
19 auto_debit_events
20 renewals
21 notifications
22 audit_logs
```

24. Development Roadmap

```
PHASE 0  Project Foundation
↓
PHASE 1  Database
↓
PHASE 2  Authentication & Security
↓
PHASE 3  Partner Management
↓
PHASE 4  Referral + Leads
↓
PHASE 5  Customers
↓
PHASE 6  Products + Sales + Subscriptions
↓
PHASE 7  Auto-Debit + Renewals
↓
PHASE 8  Dashboards
↓
PHASE 9  Notifications
↓
PHASE 10 Reports
↓
PHASE 11 Audit
↓
PHASE 12 UI/UX
↓
```

PHASE 13 Testing + Hardening
↓
PHASE 14 Production
↓
PHASE 15 Phase 2

25. Phase 0 — Exact Scope

- Create Laravel 13 project
- Configure PHP 8.3+
- Configure MySQL 8.x
- Git
- Inertia + Vue 3 + Vite
- Laravel Boost
- Project structure
- AI guidelines
- Environment conventions
- Baseline tests/build

Explicitly excluded from Phase 0: Redis, business modules, payment gateway integration and commission functionality.

26. AI Coding-Agent Rules

- Read PRD and architecture before coding.
- Never invent business requirements.
- Mark unresolved decisions as NEEDS BUSINESS CONFIRMATION.
- Follow the approved database design.
- Keep controllers thin.
- Enforce authorization server-side.
- Do not trust frontend IDs.
- Preserve historical records.
- Use transactions for multi-step business operations.
- Write tests for new functionality.
- Do not modify unrelated code.
- Do not introduce Redis without explicit approval.
- Do not choose a payment gateway without confirmation.
- Do not implement Phase 2 features in MVP.

27. Remaining NEEDS BUSINESS CONFIRMATION

- Payment gateway
- Exact subscription commercial rules
- Exact product pricing rules
- Exact Partner types beyond Main/Sub
- Exact discount rules

- Exact communication provider
- Future Customer Portal requirements
- Phase 2 commission details

28. Definition of Done

Requirement implemented

- + Database correct
- + Authorization implemented
- + Validation implemented
- + Business rules enforced
- + Error handling implemented
- + Loading/empty states implemented
- + Tests written and passing
- + No unrelated changes

29. Project Status

Area	Status
PRD	READY
Business decisions	CONFIRMED
MVP scope	DEFINED
Architecture	READY
Database / ERD	READY
Security	READY
Authorization	READY
Migration strategy	READY
Redis	NOT REQUIRED
Payment gateway	TBD
Commission	PHASE 2
Customer Portal	PHASE 2
Phase 0	READY TO START
Coding	NOT STARTED

30. Source / Basis

This master document consolidates the finalized Partner Portal requirements, business decisions, architecture, database design and implementation plan developed in this conversation. The original requirements define the Partner hierarchy, application areas, permissions, audit trail, referral attribution, renewal process and implementation guidance. No unresolved business decision has been silently converted into a requirement.